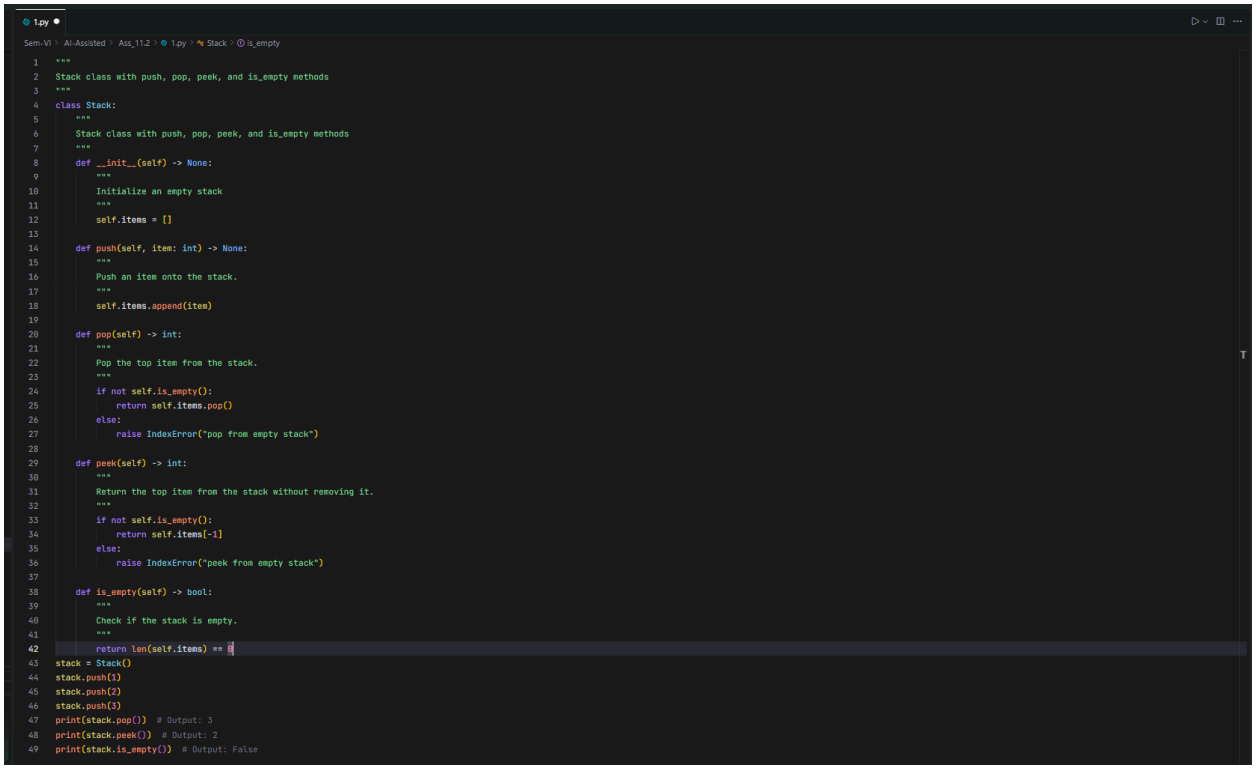
Lab Assignment 11.2

1. Stack Implementation
   Task:
   - Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

   Sample Input Code:

```python
class Stack:
    pass
```

   Expected Output:
   - A functional stack implementation with all required methods and docstrings.



```python
"""
Stack class with push, pop, peek, and is_empty methods
"""
class Stack:
    """
    Stack class with push, pop, peek, and is_empty methods
    """
    def __init__(self) -> None:
        """
        Initialize an empty stack
        """
        self.items = []

    def push(self, item: int) -> None:
        """
        Push an item onto the stack.
        """
        self.items.append(item)

    def pop(self) -> int:
        """
        Pop the top item from the stack.
        """
        if not self.is_empty():
            return self.items.pop()
        else:
            raise IndexError("pop from empty stack")

    def peek(self) -> int:
        """
        Return the top item from the stack without removing it.
        """
        if not self.is_empty():
            return self.items[-1]
        else:
            raise IndexError("peek from empty stack")

    def is_empty(self) -> bool:
        """
        Check if the stack is empty.
        """
        return len(self.items) == 0
stack = Stack()
stack.push(1)
stack.push(2)
stack.push(3)
print(stack.pop())  # Output: 3
print(stack.peek())  # Output: 2
print(stack.is_empty())  # Output: False
```

2. Queue Implementation
   Task:
   - Use AI to implement a Queue using Python lists.

   Sample Input Code:

```python
class Queue:
    pass
```

   Expected Output:
   - FIFO-based queue class with enqueue, dequeue, peek, and size methods.

2.py ● 1.py

Sem-VI > AI-Assisted > Ass_11.2 > 2.py > ...

```python
# Queue Implementation
class Queue:
    """
    Queue Implementation using Python lists
    """
    def __init__(self):
        """
        Initialize an empty queue
        """
        self.queue = []

    def enqueue(self, item: int) -> None:
        """
        Add an item to the end of the queue.
        """
        self.queue.append(item)

    def dequeue(self) -> int:
        """
        Remove and return the item from the front of the queue.
        """
        if not self.is_empty():
            return self.queue.pop(0)
        else:
            raise IndexError("dequeue from empty queue")

    def peek(self) -> int:
        """
        Return the item at the front of the queue without removing it.
        """
        if not self.is_empty():
            return self.queue[0]
        else:
            raise IndexError("peek from empty queue")

    def pop(self) -> int:
        """
        Remove and return the item from the front of the queue.
        """
        if not self.is_empty():
            return self.queue.pop(0)
        else:
            raise IndexError("pop from empty queue")
queue = Queue()
queue.enqueue(1)
queue.enqueue(2)
queue.enqueue(3)
print(queue.dequeue())  # Output: 1
print(queue.peek())  # Output: 2
```

3. Linked List

   Task:

   - Use AI to generate a Singly Linked List with insert and display methods.

   Sample Input Code:

```python
class Node:
pass
class LinkedList:
pass
```

   Expected Output:

   - A working linked list implementation with clear method documentation.

2.py 3.py 1.py

Sem-VI > AI-Assisted > Ass_11.2 > 3.py > Node

```python
# LL Implementation
class Node:
    def __init__(self, data: int):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert_at_beginning(self, data: int) -> None:
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def insert_at_end(self, data: int) -> None:
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = new_node

    def display(self) -> None:
        current = self.head
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")
```

4. Binary Search Tree (BST)
   Task:
   - Use AI to create a BST with insert and in-order traversal methods.

   Sample Input Code:

```python
class BST:
    pass
```

   Expected Output:
   - BST implementation with recursive insert and traversal methods.

4.py

Sem-VI > AI-Assisted > Ass_11.2 > 4.py > BST > _inorder_recursive

```python
# BST with insert and in-order traversal methods, BST implementation with recursive insert and traversal methods

class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None

    def insert(self, value):
        """Insert a value into the BST"""
        if self.root is None:
            self.root = Node(value)
        else:
            self._insert_recursive(self.root, value)

    def _insert_recursive(self, node, value):
        """Recursive helper method for insert"""
        if value < node.value:
            if node.left is None:
                node.left = Node(value)
            else:
                self._insert_recursive(node.left, value)
        else:
            if node.right is None:
                node.right = Node(value)
            else:
                self._insert_recursive(node.right, value)

    def inorder_traversal(self):
        """Return list of values in in-order traversal"""
        result = []
        self._inorder_recursive(self.root, result)
        return result

    def _inorder_recursive(self, node, result):
        """Recursive helper method for in-order traversal"""
        if node is not None:
            self._inorder_recursive(node.left, result)
            result.append(node.value)
            self._inorder_recursive(node.right, result)
```

5. Hash Table

   Task:

   - Use AI to implement a hash table with basic insert, search, and delete methods.

   Sample Input Code:

```python
class HashTable:
    pass
```

   Expected Output:

   - Collision handling using chaining, with well-commented methods.

4.py 6.py 5.py

Sem-VI > AI-Assisted > Ass_11.2 > 5.py > HashTable > delete

```python
# Hash table with basic insert, search, and delete methods, with well-commented methods.
class HashTable:
    """A simple hash table implementation using chaining for collision resolution."""

    def __init__(self, size=10):
        """Initialize the hash table with a specified size."""
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """Compute the hash value for a given key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert a key-value pair into the hash table."""
        index = self._hash(key)
        bucket = self.table[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
                return
        bucket.append((key, value))

    def search(self, key):
        """Search for a value by its key."""
        index = self._hash(key)
        bucket = self.table[index]
        for k, v in bucket:
            if k == key:
                return v
        raise KeyError(key)

    def delete(self, key):
        """Delete a key-value pair from the hash table."""
        index = self._hash(key)
        bucket = self.table[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                del bucket[i]
                return
        raise KeyError(key)
```

6. Graph Representation

   Task:

   - Use AI to implement a graph using an adjacency list.

   Sample Input Code:

```python
class Graph:
    pass
```

   Expected Output:

   - Graph with methods to add vertices, add edges, and display connections.

4.py 5.py

Sem-VI > AI-Assisted > Ass_11.2 > 5.py > Graph > display_connections

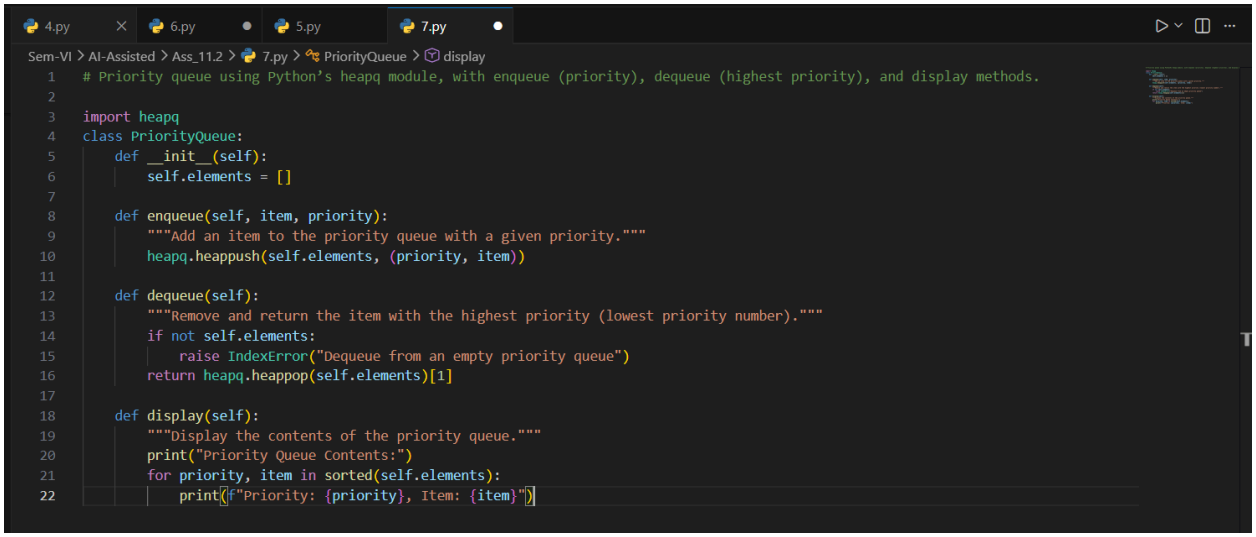
```python
# Graph using an adjacency list, with methods to add vertices, add edges, and display connections.

class Graph:
    def __init__(self):
        self.adjacency_list = {}

    def add_vertex(self, vertex):
        """Add a vertex to the graph"""
        if vertex not in self.adjacency_list:
            self.adjacency_list[vertex] = []

    def add_edge(self, vertex1, vertex2):
        """Add an edge between two vertices in the graph"""
        if vertex1 in self.adjacency_list and vertex2 in self.adjacency_list:
            self.adjacency_list[vertex1].append(vertex2)
            self.adjacency_list[vertex2].append(vertex1)  # For undirected graph

    def display_connections(self):
        """Display the connections of the graph"""
        for vertex, edges in self.adjacency_list.items():
            print(f"{vertex}: {', '.join(edges)}")
```

7. Priority Queue
   Task:
   - Use AI to implement a priority queue using Python's heapq module.

   Sample Input Code:

```
class PriorityQueue:
    pass
```

   Expected Output:
   - Implementation with enqueue (priority), dequeue (highest priority), and display methods.




```python
# Priority queue using Python's heapq module, with enqueue (priority), dequeue (highest priority), and display methods.

import heapq
class PriorityQueue:
    def __init__(self):
        self.elements = []

    def enqueue(self, item, priority):
        """Add an item to the priority queue with a given priority."""
        heapq.heappush(self.elements, (priority, item))

    def dequeue(self):
        """Remove and return the item with the highest priority (lowest priority number)."""
        if not self.elements:
            raise IndexError("Dequeue from an empty priority queue")
        return heapq.heappop(self.elements)[1]

    def display(self):
        """Display the contents of the priority queue."""
        print("Priority Queue Contents:")
        for priority, item in sorted(self.elements):
            print(f"Priority: {priority}, Item: {item}")
```

8. Deque
   Task:
   - Use AI to implement a double-ended queue using collections.deque.

   Sample Input Code:

```
class DequeDS:
    pass
```

   Expected Output:
   - Insert and remove from both ends with docstrings.

4.py 6.py 5.py 7.py 8.py

Sem-VI > AI-Assisted > Ass_11.2 > 8.py > Deque > display

```python
"""
Double-ended queue using collections.deque with Insert and remove from both ends with docstrings
"""

from collections import deque
class Deque:
    def __init__(self):
        """Initialize an empty deque."""
        self.deque = deque()

    def insert_front(self, item):
        """Insert an item at the front of the deque."""
        self.deque.appendleft(item)

    def insert_rear(self, item):
        """Insert an item at the rear of the deque."""
        self.deque.append(item)

    def remove_front(self):
        """Remove and return the item from the front of the deque."""
        if not self.deque:
            raise IndexError("Remove from an empty deque")
        return self.deque.popleft()

    def remove_rear(self):
        """Remove and return the item from the rear of the deque."""
        if not self.deque:
            raise IndexError("Remove from an empty deque")
        return self.deque.pop()

    def display(self):
        """Display the contents of the deque."""
        print("Deque Contents:", list(self.deque))
```